HACK IN HILLS ’26

Problem Statements & Challenge Tracks

01. Vision-Based Autonomous Robot

Problem Statement

Build an autonomous robot that uses an AI-powered vision system to understand its surroundings, identify relevant objects and people, and navigate autonomously toward a given objective while detecting and avoiding obstacles.

Suggested Technical Directions

- Object detection
- Image classification
- Scene understanding
- Vision-language models
- Motor control
- Sensors and perception
- Autonomous navigation
- Obstacle detection and avoidance
- Real-time decision making

Challenge

Create a functional robotic prototype demonstrating how AI vision can be combined with robotics for autonomous perception, navigation, and decision-making in the physical world. Teams may demonstrate the system in a real or simulated environment.

02. Onchain Finance & Trading

Problem Statement

Build a fully onchain financial product that leverages fast blockchain settlement to create innovative and transparent trading or lending markets without relying on centralized offchain infrastructure.

Possible Directions

- Fully onchain order books with matching and settlement handled onchain.
- Perpetual futures markets with funding rates updated every block.
- Undercollateralized lending using a user’s onchain credit history to determine lending terms.
- Onchain market-making systems that provide liquidity and dynamically respond to market conditions.

Challenge

Create a functional prototype demonstrating how blockchain-native infrastructure can enable financial products that are difficult, inefficient, or less transparent to build using traditional centralized systems. Clearly demonstrate the onchain components, transaction flow, and core financial mechanism.

03. Commodity Derivatives Intelligence

Problem Statement

Build a data product using India's public commodity futures data to identify and analyze relative pricing differences between contracts representing the same underlying metal.

MCX lists gold futures in multiple contract sizes, including GOLDM, GOLDTEN, GOLDGUINEA, and GOLDPETAL. Because these contracts represent the same underlying metal, their normalized prices should be closely related. The challenge is to understand, measure, and responsibly analyze the differences between them.

Possible Directions

Cross-Contract Relative Value

- Normalize contract size, quotation base, and purity.
- Identify when one contract appears unusually cheap or expensive relative to another.

Term Structure & Carry Analytics

- Analyze the futures curve.
- Separate mechanical roll-down toward expiry from genuine changes in the curve.

Walk-Forward Backtesting

- Replay historical contracts day by day without look-ahead bias.
- Account for transaction costs and thin-day liquidity.

Trader-Facing Intelligence

- Build dashboards or alerts that highlight meaningful opportunities.
- Keep alerts quiet when there is no meaningful signal.
- Attribute performance to the strategy rather than simply to gold-price movement.

Contract Lifecycle Planning

- Track listing dates, liquidity development, tender periods, and expiry.
- Place every intended entry and exit inside the relevant contract calendar.

Data Sources

MCX Daily Bhavcopy: https://www.mcxindia.com/market-data/bhavcopy

MCX Gold Contract Specifications: https://www.mcxindia.com/products/bullion/gold

The daily Bhavcopy provides Symbol, Date, ExpiryDate, Open, High, Low, Close, Volume, and OpenInterest. The data is publicly available and does not require an API key.

Important Data Considerations

- Validate the returned Date against the requested date. MCX may return the most recent available trading day for an unrecognized, holiday, malformed, or future date.
- Requests use DD/MM/YYYY, while the response Date uses MM/DD/YYYY. ExpiryDate uses a compact format such as 04SEP2026, and Symbol values may be space-padded.
- Track contracts by expiry_date rather than using a continuous near-month series. A near-month series changes contracts at expiry and can create artificial price jumps.

- Settlement prices are official exchange settlement values and should not automatically be treated as executable fill prices, particularly for thin contracts.
- Volume is not the same as market depth or available liquidity.
- GOLDTEN is available only from its 2025 listing period onward; comparisons involving GOLDTEN therefore have a shorter history.

Contract Mechanics

Contract	Trading Unit	Price Quoted Per	Purity	Expiry Window
GOLDM	100 g	10 g	995	3rd–5th
GOLDTEN	10 g	10 g	999	27th–31st
GOLDGUINEA	8 g	8 g	999	27th–31st
GOLDPETAL	1 g	1 g	999	27th–31st

Challenge

Create a functional prototype that transforms exchange settlement data into a defensible analytical signal or intelligence product and validates the approach using unseen historical data. Where a strategy or signal is presented, report performance after relevant costs using the prices of the contracts actually held, and clearly separate strategy performance from the impact of the underlying gold price.

A rigorous demonstration that no persistent edge survives costs is also a valid analytical outcome. The objective is analytical rigor rather than generating a signal every day.

04. Securing Adversarial AI Safety Testing Infrastructure — Bayora

Introduction

AI systems are increasingly being deployed in areas where reliability, security, and safety are critical. As these systems become more capable, robust adversarial testing and safety validation infrastructure is essential.

Bayora is an AI safety validation platform built around a red-team / blue-team model. The red team systematically tests language models using adversarial prompts, jailbreak sequences, and injection strategies, while the blue team develops defensive countermeasures. Together, they form a continuous adversarial testing loop intended to identify safety failures before deployment.

Problem Statement

Bayora's red team, blue team, and client LLM currently operate within a shared Docker-based cloud sandbox. This creates a fundamental security challenge: adversarial and defensive operations must run simultaneously, while the validity of each safety finding depends on strong isolation between the tenants and the model under test.

- Red-team attack payloads must not be observable by the blue team before a test concludes.
- Blue-team defensive logic must not be exposed or inferable by red-team tooling.
- The client LLM must remain clean and uncontaminated throughout the evaluation.
- Safety findings must not be invalidated by environmental leakage or cross-tenant interference.

Core Question

How can you build a shared environment where adversaries, defenders, and the model under test operate simultaneously without exposing their sensitive state or creating exploitable side channels?

Recommended Technical Verticals

1. Container & Sandbox Isolation

Explore stronger execution boundaries using Linux namespaces, seccomp profiles, cgroup partitioning, rootless containers, and hardened container-runtime patterns.

2. Network Segmentation & Traffic Control

Design micro-segmented network topologies where communication between red-team, blue-team, and model workloads is strictly controlled and policy-enforced.

3. Access Control & Secrets Management

Develop capability-based or attribute-based access models that restrict access to attack payload repositories, defensive logic, classifier weights, and evaluation artifacts.

4. Audit Trails & Forensic Provenance

Create tamper-evident and cryptographically verifiable records so safety findings can be independently reconstructed and validated.

5. Resource Governance & Fairness

Prevent CPU, memory, I/O, and timing effects from distorting evaluation results or creating exploitable side channels.

6. LLM-Specific Threat Surface

Address risks such as shared KV-cache side channels, prompt-context leakage, model-output isolation, inference-time data exfiltration, and cross-session contamination.

7. Observability & Anomaly Detection

Detect cross-boundary access, policy violations, and suspicious activity without turning the monitoring layer into an information-leakage channel.

Evaluation Objectives

01 — Isolate Without Breaking Function

Enforce strong boundaries between the red team, blue team, and model while preserving real-time functionality and avoiding latency that invalidates evaluation results.

02 — Prevent Information Leakage

Prevent attack artifacts, defensive strategies, and model state from being directly or indirectly exposed across boundaries, including through potential side channels.

03 — Guarantee Audit Integrity

Maintain a verifiable and tamper-evident record of evaluation activity so safety findings can be independently reconstructed.

04 — Build for Deployability

Deliver a working proof of concept that can run on standard cloud infrastructure and existing container runtimes without proprietary hardware dependencies.

05 — Address LLM-Native Threats

Go beyond conventional infrastructure security and address risks specific to shared AI inference environments.

06 — Quantify Residual Risk

Provide a clear threat model describing what the architecture protects against, what it does not protect against, conditions under which guarantees may fail, and future improvements required.

Suggested Datasets & Resources

AdvBench: https://github.com/llm-attacks/llm-attacks

JailbreakBench: https://jailbreakbench.github.io

HarmBench: https://www.harmbench.org

ToxiGen: https://github.com/microsoft/ToxiGen

Gandalf / Lakera Red Teaming Dataset: https://huggingface.co/datasets/Lakera/gandalf_ignore_instructions

NVD / CVE Database: https://nvd.nist.gov/vuln/search

DARPA Transparent Computing / Provenance Resources:
https://github.com/darpa-i2o/Transparent-Computing

Anthropic HH-RLHF: https://huggingface.co/datasets/Anthropic/hh-rlhf

Final Challenge

Build a working proof of concept for secure adversarial AI testing infrastructure that demonstrates meaningful isolation, prevents cross-team information leakage, preserves auditability, and clearly communicates the security guarantees and residual risks of the architecture.

Deep expertise in large language models is not required. Teams with backgrounds in systems engineering, cloud infrastructure, cybersecurity, distributed systems, and container security are strongly encouraged to participate.